

Offline Sinhala Speech to Text Model Training Guide Using Wav2Vec2

This is a Sinhala Automatic Speech Recognition (ASR) system capable of converting Sinhala speech into text. A pre trained Wav2Vec2 model was finetuned using a custom Sinhala speech dataset consisting of audio recordings and manually verified transcripts.

The final trained model can be used locally(offline) to generate Sinhala text from audio recordings.

Development Environment

Platform: Google Colab

Hardware: GPU Runtime (Tesla T4)

Step 1:

Open Google Colab [Google Colab](#)

Click File → New Notebook

Enable GPU

- Click **Runtime** (top menu)
- Select **Change runtime type**
- Under **Hardware accelerator**, choose **GPU**
- Click **Save**
- Verify That GPU Is Enabled. Run this code in a cell:

```
import torch

print("GPU Available:", torch.cuda.is_available())
print("GPU Name:", torch.cuda.get_device_name(0))
```

If successful, you'll see something like:

GPU Available: True

GPU Name: Tesla T4

Python Libraries:

- **Transformers**: Used for text generation, translation, summarization, question answering, and speech tasks.
- **Datasets**: Makes it easy to download, load, preprocess, and manage machine learning datasets.
- **Accelerate**: Simplifies training models on GPUs, multiple GPUs, or TPUs
- **TorchAudio**: Handles audio data in PyTorch. Used for loading audio files, preprocessing audio, and building speech recognition systems.
- **Librosa**: Analyzes audio signals and extracts features such as MFCCs, spectrograms, tempo, pitch, and frequency information.
- **SoundFile**: Reads and writes audio files such as WAV, FLAC, and OGG. Often used together with Librosa.
- **SentencePiece**: Converts text into tokens (subwords) that AI models can understand. Widely used in models like T5 and multilingual language models.

Step 02:

```
!pip uninstall -y torchao  
  
!pip install -q \  
transformers \  
datasets \  
accelerate \  
peft \  
torchaudio \  
librosa \  
soundfile \  
sentencepiece
```

Dataset Preparation

The original dataset contains,

- Long audio recordings (1–5 minutes) in wav format. Audios should be in wav format if not convert them into wav format.
- Corresponding transcripts

Audio Chunking

Long recordings are difficult for speech recognition models. Therefore, every audio file was divided into 20 second chunks.

Example

Original Audio: audio1.wav (Duration: 4 minutes)

Generated Chunks: Each chunk is approximately 20 seconds long.

- sample_0_0.wav
- sample_0_1.wav
- sample_0_2.wav
- ...

Transcript Correction

After chunking, all chunks were manually reviewed. Each audio chunk was matched with its correct transcript segment.

A new CSV file was created and save this file as transcripts.csv

audio	text
sample_0_0.wav	ආයුබෝවන් මම ඊරිනා. මට පුළුවනි ඔබට සහය වන්න?.....
sample_0_1.wav	871 කියන නම්බර් එකද? ඔව් ඔව්. ඒක වැරදි කියලා පොඩ්ඩක් බලන්නකෝ ඇන්ට්ට් ද?
sample_0_2.wav	හරි. ඔව් රැඳී සිටියාට ස්තූතියි. මැඩම් දැනට කනෙක්ෂන් එක බ්ලෑපත් නිසා ඩිස්කනෙක්ට් වෙලා තියෙන්නේ

Step 03: Dataset Structure in your local pc

```
dataset for Wav2vec/  
  audio 20sec/  
    sample_0_0.wav  
    sample_0_1.wav  
    sample_0_2.wav  
    ...  
  transcripts.csv
```

Step 04: Upload Dataset to Google Colab

Upload the dataset ZIP file.

```
from google.colab import files  
  
uploaded = files.upload()
```

Select:

dataset for Wav2vec.zip

Step 05: Extract Dataset

```
import zipfile  
  
import os  
  
with zipfile.ZipFile("dataset for Wav2vec.zip", "r") as zip_ref:  
    zip_ref.extractall("/content/dataset")  
  
print("EXTRACTED")  
print(os.listdir("/content/dataset"))
```

Step 06: Load Transcript File

```
import pandas as pd  
  
import os  
  
BASE = "/content/dataset/dataset for Wav2vec/audio 20sec/"  
  
df = pd.read_csv(  
    "/content/dataset/dataset for Wav2vec/transcripts.csv"  
)  
  
df["audio"] = df["audio"].apply(  
    lambda x: BASE + str(x)  
)  
  
print(df.head())  
  
print("Total Rows:", len(df))
```

Step 07: Verify Audio Files

```
df["exists"] = df["audio"].apply(os.path.exists)

print(df["exists"].value_counts())
```

Expected Output: (ex;)

```
True    191
False     1
```

Step 08: Clean Dataset

```
import re

df = df[df["exists"] == True].copy()

def clean_text(text):
    text = str(text)
    text = re.sub(r"\s+", " ", text)
    text = re.sub(
        r"[\u0D80-\u0DFFa-zA-Z0-9 .,?]",
        "",
        text
    )
    return text.strip()

df["text"] = df["text"].apply(clean_text)

df = df[df["text"].str.len() > 5]

print("FINAL CLEAN DATASET:", len(df))
```

Expected Output:

```
FINAL CLEAN DATASET: 189
```

Step 09: Load Wav2Vec2 Model

```
import torch
import librosa
import numpy as np

from datasets import Dataset

from transformers import (
    Wav2Vec2Processor,
    Wav2Vec2ForCTC,
    TrainingArguments,
    Trainer
)

MODEL_NAME = "janiduchamika/wav2vec2-xls-r-300m-sinhala-general-185k"

processor = Wav2Vec2Processor.from_pretrained(
    MODEL_NAME
)

model = Wav2Vec2ForCTC.from_pretrained(
    MODEL_NAME,
    low_cpu_mem_usage=True
)

model.config.use_cache = False
model.gradient_checkpointing_enable()

print("MODEL LOADED")
```

Expected Output:

MODEL LOADED

Step 10: Convert DataFrame to Dataset

```
dataset = Dataset.from_pandas(df)

dataset = dataset.remove_columns(
    ["exists", "__index_level_0__"]
)

print(dataset)
Expected Output:
Dataset({
  features: ['audio', 'text'],
  num_rows: 189
})
```

Step 11: Audio Processing

```
def process_audio(path):

    audio, sr = librosa.load(
        path,
        sr=16000
    )

    audio = audio / (
        np.max(np.abs(audio)) + 1e-7
    )

    return audio
```

Step 12: Prepare Dataset

```
def prepare_batch(batch):

    audio = process_audio(
        batch["audio"]
    )

    inputs = processor(
        audio,
        sampling_rate=16000
    )

    batch["input_values"] = inputs.input_values[0]

    batch["labels"] = processor.tokenizer(
        str(batch["text"]))
    ).input_ids

    return batch

dataset = dataset.map(
    prepare_batch
)

print(dataset)
```


Step 13: Create Data Collator

```
import torch

def data_collator(features):

    input_values = [
        torch.tensor(f["input_values"])
        for f in features
    ]

    labels = [
        torch.tensor(f["labels"])
        for f in features
    ]

    batch = {}

    batch["input_values"] = torch.nn.utils.rnn.pad_sequence(
        input_values,
        batch_first=True
    )

    batch["labels"] = torch.nn.utils.rnn.pad_sequence(
        labels,
        batch_first=True,
        padding_value=-100
    )

    return batch

print("Data collator ready")
```

Step 14: Configure Training

```
training_args = TrainingArguments(  
    output_dir="./wav2vec2-sinhala-final",  
  
    per_device_train_batch_size=1,  
  
    gradient_accumulation_steps=8,  
  
    learning_rate=1e-4,  
  
    num_train_epochs=10,  
  
    fp16=True,  
  
    save_steps=50,  
  
    logging_steps=10,  
  
    eval_strategy="no",  
  
    dataloader_pin_memory=False,  
  
    remove_unused_columns=False,  
  
    report_to="none"  
)
```

Step 15: Create Trainer

```
trainer = Trainer(  
    model=model,  
    args=training_args,  
    train_dataset=dataset,  
    eval_dataset=dataset,  
    data_collator=data_collator  
)  
print("Trainer ready")
```

Step 16: Train the Model

```
trainer.train()
```

Training Duration:

Approximately 30 minutes on Tesla T4 GPU

Step 17: Save the Trained Model

```
trainer.save_model(  
    "/content/wav2vec2-sinhala-final-model"  
)  
  
processor.save_pretrained(  
    "/content/wav2vec2-sinhala-final-model"  
)  
  
print("Model saved")
```

Step 18: Create ZIP File

```
!zip -r \
wav2vec2-sinhala-final-model.zip \
/content/wav2vec2-sinhala-final-model
```

Step 19: Download the Model

```
from google.colab import files
files.download(
    "wav2vec2-sinhala-final-model.zip"
)
```

The downloaded ZIP file contains the trained Sinhala Speech-to-Text model and can be used locally on a Windows PC for transcription.